Scalability & Performances Evaluation of LOCARN: Low Opex and Capex Architecture for Resilient Networks

Damien Le Quéré*, Christophe Betoule*, Rémi Clavier*, Yassine Hadjadj-Aoul†,
Adlen Ksentini† and Gilles Thouénon*
*Orange Labs, Lannion, France
Email: firstname.lastname@orange.com
†IRISA, University of Rennes I, France
Email: firstname.lastname@irisa.fr

***Abstract*—This paper proposes LOCARN: an alternative network architecture providing a packet connectivity layer, which is able to self-adapt its routing paths to both the effective traffics fluctuations and network resources changes. Moving close to a global maximization of available resources usage and assuming high resiliency under failures, this radical architecture focuses on architectural components coupling simplicity and plug-and-play guidance. Through analysis and computer simulation, several performance metrics focusing on scalability are evaluated.**

***Keywords*—packet transport network, flat network architecture, self-* properties, scalability evaluation**

I. Introduction

Traditionally, telecommunication network operators extensively use a centralized approach to manage their transport networks (i.e. their metro/core segments architectures and technologies) . Typically, a central entity called the Network Management System is controlling Network Elements through embedded components called Element Management Systems within an administrative domain (NMS, NEs and EMSs). Over the last decade, important activities in research and standardization have led to the automation of transport network architectures through the introduction of distributed control planes. In practice, two main trends are possible today for operators: the automation of packet switched transport planes and the automation of optical circuit switching networks. The first perspective actually consists in the deployment of the IP/MPLS technology with its intrinsic "automation suite" standards [1]–[5]. Some research efforts toward Ethernet based technologies also exist and aim to provide large scale automated packet networks, under the name of "layer 2 routing" [6]–[8]. The second trend, referred as ASONs (Automatically Switched Optical Networks) consists, in practice, in the use of the MPLS protocol suite generalization (GMPLS) [9], used as control plane for optical architectures. In addition, in recent years the PCE architectures [10] propose to introduce dedicated entities to achieve complex transport decisions directly in the network, including ones that operators traditionally performed offline like resources planning and optimization. Although the automation of operator Transport Networks is really progressing, it goes with a more complexe design.

978-1-4799-5350-9/14/$31.00 © 2014 IEEE

In 2010, an innovative packet transport solution named LOCARN (i.e. Low Opex and Capex Architecture for Resilient Networks) has been proposed in the context of the European TIGER2 project [11] whose one of its objectives aimed at making packet based transport network solution evolving towards higher manageability. To that end, the project has explored several proposals to improve the networks' control and management by focusing on the autonomic networking aspects that would allow to master complexity [12], [13]. LOCARN is born from the observation that trend-setting developments of automated Transport Networks involves a significant collection of components and mechanisms in nodes (protocols, tables, path computation module, label/bandwidth allocation, mechanisms for protection). The LOCARN architecture proposes to revisit the way to think network management automation by a new design of its (own) data and control plane in order to guaranty a high level of *simplicity*, *self-adaptation* and *resiliency*. The unpleasant aspect to reach a high level of simplicity, dynamicity and adaptability lies on the amount of non-data packets generated that may possibly present some scalability issues. The objective of this paper is to go further by studying LOCARN performances, concentrating particularly on the scalability in the perspective of an application to transport networks.

The paper is organized as follows. Section II presents the architecture through its technical overview, the analysis of its benefits, and the discussion around the overhead issues. Section III estimates the architecture overheads cost by analysis. Section IV presents simulation results over an existing transport network topology. Finally the section V concludes this paper.

II. The LOCARN Architecture

A. Technical Overview

LOCARN is a flat packet network architecture providing point-to-point bidirectional communications through the definition of its own data plane and control plane mechanisms, in a complete agnostic way with regards to the IP stack. The functional architecture (illustrated in Fig.1) is composed by two kinds of functional nodes: (i) *Edges Nodes* (ENs) that constitute ingress/egress nodes of a LOCARN transport domain

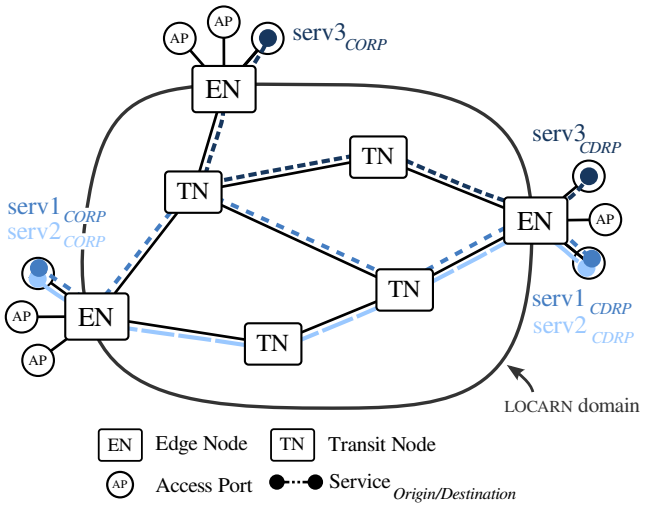


Fig. 1. The LOCARN functionnal architecture on a small example

(ii) *Transit Nodes* (TNs) that solely operate packet forwarding. The functional architecture also defines: (i) *Access Ports* (APs) that are external ENs ports (they can be physical or logical ports according to the implementation guidance) (ii) *Services* that are end-to-end bidirectional channels[1]. Services provide customer data with the transport of its client information between two APs across the domain. They are composed by two Reference Points (RPs) sharing a common service identifier: a Connection Origin RP (CORP) and a Connection Destination RP (CDRP). The only management operations required in LOCARN is registration/unregistration of RPs using unequivocal services names among the domain. Then LOCARN is in charge of the establishment, maintenance and adaptation of declared services over time.

The LOCARN solution concept relies on three network mechanisms:

Autoforwarding – Autoforwarding designates a packet relaying mechanism that uses exclusively information contained in the packet itself for the determination of the next hop interface. In LOCARN, the data packets are autoforwarded from end-to-end by carrying in headers the complete sequence of output ports to cross from the origin to the destination (including the AP as the last port). Autoforwarding leads to a drastic simplification of the data plane technical requirements, both in terms of computation and memorizing. Indeed, the autoforwarding header is just added at data packets encapsulation in the ingress EN whereas successive TNs forward packets by obtaining the output port directly from this header. A sequence cursor is incremented at each step of the course to indicate the header position to read. An autoforwarding data plane does not require to store any Forwarding Information Base (FIB) in TNs neither a fortiori to perform any table lookup operation.

Enhanced flooding – To obtain the autoforwarding header information included at the data packets encapsulation, an EN runs a source routing process for each CORP registration. A LOCARN source routing process can be decomposed into six steps: (i) a service path request is flooded across the network; (ii) over time, some request messages reach the EN where the corresponding CDRP is registered (arriving from distinct end-to-end paths); (iii) for each request message, the "destination EN" returns an answer message to the "origin EN" [2]. Each answer message is carrying a distinct path autoforwarding information to the origin EN (each request messages memorized the crossed output ports along its course); (iv) along the backward course, answer messages take the opportunity to collect statistics about the crossed interfaces (bandwidth usage, queuing usage…); (v) then the origin EN typically receives numerous answer packets which each one provides an autoforwarding port sequence associated with end-to-end statistics about the path. Let us notice that loops are intrinsically prevented during the flooding propagation by deleting packets that followed a cycle; (vi) finally the EN origin selects the best path by successive comparisons between the arriving path and the one that is currently used. In LOCARN, source routing is thus purely achieved through paths *discovery* and *selection*. No routing algorithm is involved, no routing computation or routing tables either, and no convergence times: a path has changed from the moment that the encapsulation function has been modified.

The decisive specificity of LOCARN routing lies on the fact that: (i) a routing process is performed for each service – "serv1" and "serv2" illustrated this in Fig.1 ; (ii) among the (large) amount of discovered paths, a service will typically favors the less used paths. To do so, the service's path selection is partially or totally based on the discovered paths' meta-information collected (i.e. interfaces statistics about bandwidth consumption and queuing loads) which actually reflects the level of effective resources usage along the path[3]; (iii) for each service, the source routing steps are relaunched periodically to adapt the active path over time to any kind of noticeable evolution between the endpoints (referred as "path-optimizations"). Thus, on one hand each service self-adapts periodically according to its perception of the network state between its endpoints; and on the other hand a service adaptation is itself noticeable by other services within the domain, and may trigger their own adaptation[4]. Finally, over successive path-optimizations, LOCARN tends hollistically toward a global, opportunistic and very adaptative distribution of client traffics among the domain without involving any complexity in the achitectural design. In-depth studies regarding LOCARN dynamic behaviors are out of the scope of this document – both positive behaviors like overall path arrangements and negative aspects like possible erratic behaviors.

End-to-end fault detection – The periodical relaunch of

[1]the LOCARN architecture forms a layer network in the sense of the ITU-T G.805 recommendation: "A topological component that represents the complete set of access groups of the same type which may be associated for the purpose of transferring information"; in the ITU transport terminology, a LOCARN service is a point-to-point *transport entity*, i.e.: "an architectural component which transfers information between its inputs and outputs within a layer network"

[2]answers are autoforwarded along the corouted reverse paths from destination to origin, each request messages memorized the crossed input ports along its course

[3]several routing policies can be implemented. The use of the available bandwidh criteria tends to balance loads for example whereas the end-to-end delay criteria can be used to provide adapted path for clients traffics sensitive to delay (on its side, a path end-to-end delay is obtained by noticing the Round Trip Time (RTT) at the path answer arrival time).

[4]a service adaptation leads to a more or less significant redirection of the client traffic which impacts interfaces statistics that will be collected by future services' discoveries

source routing steps exposed above make services able to recover if the connectivity has been lost for any reason. Yet, the optimization periodicity is envisioned at least around several seconds making the recovery time relatively long. With regards to the operators Transport Networks requirements in terms of resiliency, each LOCARN active path is continously checked by very frequent round trips of specific small OAM packets (i.e. Operation And Maintenance); the lack of three OAM answers at the CORP means that a fault has occured. These packets exchanges, initiated by CORPs, are strictly transfered like data packets with autoforwarding. The interval of OAM sends determines the detection time, it must be envisionned around several milliseconds to reach carrier grade networks.

B. LOCARN Benefits: Properties & Behaviors

We assume that in an operator viewpoint, the overall interest of their Transport Networks automation are: (i) the simplification of connectivity provisioning through limiting complex planning and design processes; (ii) the efficiency of network resource's utilization especially through dynamic shared restoration in replacement of dedicated static protection; (iii) rapid service turn-up through the timely support of bandwidth on-demand services which allow some new revenue opportunities. In such perspectives, LOCARN exhibits some advantageous properties and behaviors:

LOCARN is simple: LOCARN proposes simple nodes design based on minimal computation and storage: low states in edge nodes and no state at all in transit nodes. Hence the architecture is easy to implement into software or hardware components with low technical requirements. Furthermore LOCARN is simple to operate: the addition of new communications simply consist in the declaration of the appropriate services Reference Points. Once it is established, each service is included in the network domain adaptation process without requiring any planning-configuring step. The LOCARN operational simplicity is known as "plug-and-play", because the network is ready to work as soon as the devices are plugged onto the underlying layer whereas a service is ready to transmit immediately from the moment its two endpoints have been declared. The simplicity is besides a mean to reduce the financial costs: the nodes low technical prerequisites involve a minimum CAPital EXpenditure (CAPEX) whereas the operational simplicity leads to minimize the OPerationnal EXPenditure (OPEX). Finally, the data plane simplicity constitutes a significant potential for energy saving because both Forwarding Information Bases (FIBs) and look-up operations are completely eliminated for packet switching along the transit.

LOCARN assumes resources resiliency: LOCARN copes with resources failures, ensuring the continuity of client information transit with the minimum of impact. The LOCARN resiliency relies on a purely reactive restoration process without any preliminary configuration of any protection path. On a service fault detection (related to the settings of OAM exchanges), the CORP simply re-launch the source routing processes in order to restore the path. This reactive approach is possible because the path discovery is very quick (a path between an origin and a destination is basically obtain during its Round Trip Time). Doing so, LOCARN avoids the waste of network transmission capacity in regard to the static dedicated protection, moreover it is intrinsically able to cope with multiple failure uses cases, which is a major distinction with protection mechanisms used in MPLS/GMPLS based solutions (see "Fast Reroute" [2]). Since an end-to-end path can be discovered toward the destination, it constitutes a potential restoration path. Hence, in LOCARN the simple resource addition (links, nodes) within a domain increases the network robustness and the services availability. In terms of operational implications, this completely reactive approach greatly simplify maintenance by preserving the "plug-and-play" property. For example, a node isolation that is usually an heavy and error prone operation in transport technologies simply consists in deplug it and replug it. The degree of resiliency is tunable according to the frequency of OAM exchanges.

LOCARN assumes clients dynamicity: LOCARN has been conceived to handle dynamic clients transportation demands. Indeed, whether because of an irregular bandwidth demand of services or because of an irregular arrival or departure of these latter, routing paths are adapting among the LOCARN domain according to the effective consumption observed. Many communications can established and released without involving adaptations if they cause a minimum impact on other services whereas the ponctual need for bandwidth of one service may involve several paths shifts for example. Simply, a LOCARN layer constantly adapts all paths to satisfy both evolutions of the transportation demands and evolution of the available resources. The degree of dynamicity is tunable according to the frequency of path optimizations processes.

LOCARN is a suitable base for Autonomic Networking: According to the terminological distinction exposed in [14] (beginning of section II), we prefer the "automatic" that "autonomic" term to qualify LOCARN in so far that in the actual architecture, no High Level Objectives (HLOs) are externally defined to pilot the path adaptations criteria. However, an Autonomic Network Management System (ANMS) could be introduce quite easily for the coordination or ENs decisions on the base of informations they have collected over time. This way, LOCARN can be the mean to introduce autonomic networking principles in an incremental way.

C. LOCARN issues: Overheads and Scalability

As exposed in introduction, the LOCARN's counterparty to its simplicity and dynamicity is the amount of non-client messages generated over time. Indeed, unlike other protocols, the LOCARN overhead is not only related to the network dimensions, but it also depends on the amount of services present. We distinguish three kinds of overheads corresponding to the three basic LOCARN mechanisms in so far each one involves the transfer of non client information.

Data plane overhead: Because of autoforwarding, the LOCARN data packets header's size depends on the path's lengths, which presents a priori a scalability issue. However, with the LOCARN specific framing scheme, data headers are actually smaller than common packet transport technologies. Indeed, in LOCARN a data header solely contains output ports from the source to the destination but it includes no node identifier neither for source, destination or intermediate nodes.

In practice, for a 20 hops path[5] that needs to store 20 output ports, if the implementation assumes for example that the identifier of transit ports is encoded on 1 byte (allowing 256 transit ports per Transit Node), the header remains smaller than a MAC-in-MAC encapsulation of Carrier Ethernet frames (802.1ah) and it remains equivalent to five MPLS label stack entries [15]. Hence, data headers should not be forsaken for the accurate calculation of the total overhead in a LOCARN, yet it is not a critical issue beside the non-data message generation as we shall see hereafter.

Enhanced flooding's overhead: The flooding overhead is definitively the point studied here. Let us first notice that it can not be envisioned to let the flooding process terminates by itself since the number of generated messages would exceed a reasonable number, even for small networks. In LOCARN the floods propagation is simply limited by introducing a Time To Live (TTL) mechanism to path requests messages. This approach raises the question of the TTL tuning. We easily understand that the TTL must be at least equal to the shortest path(s) length between an origin and a destination to allow the establishment of a connectivity; whereas each TTL increment increases the amount of path discovered whose the beneficial consequence is a higher potential for path optimization and for recovery. For simplification, we consider in following sections a global TTL applied to all services path discoveries. The selected value is comprised between the network diameter and a reasonable upper bound to be estimated.

OAM overhead: The OAM exchanges between service's end-points is another critical factor of overhead cost in LOCARN due their high frequency. Indeed, to meet high resiliency level, OAM messages must be sent with high frequency (i.e. around an interval of 10ms or 15ms for high resiliency requirements under 50ms).

D. Related Works

The first LOCARN conceptual ancestor is, in the historical order, the Dynamic Source Routing protocol [16]. Conceived for the Mobile Adhoc NETworks, DSR is a routing protocol that aims to minimize the routing messages overhead by considering the resources scarcity of the wireless environment. On the contrary to proactive table driven IP routing protocols (e.g. OSPF) that continuously diffuses control packets among the network, DSR achieves reactive routing from the source on demand. The succession of nodes to follow until the destination are collected at the source. Then, the whole path to the destination is specified in packets specific headers for their routing from end-to-end.
The second related work that is conceptually close to LOCARN is the APLASIA architecture [17]. APLASIA work is inspired from the LOCARN initiative and is notably based on the same primary concepts, namely autoforwarding and routing based on paths discoveries by means of network floods. Yet, in the APLASIA design, those mechanisms are used in a different way, providing ultimately very different properties and behaviors than LOCARN. Among other differences, the Adaptive Probabilistic Flooding [18] algorithm is employed to minimize the message generation involved by flooding processes (reducing at the same time the amount of discovered paths).

[5]that can be considered as a very long path across a single transport domain

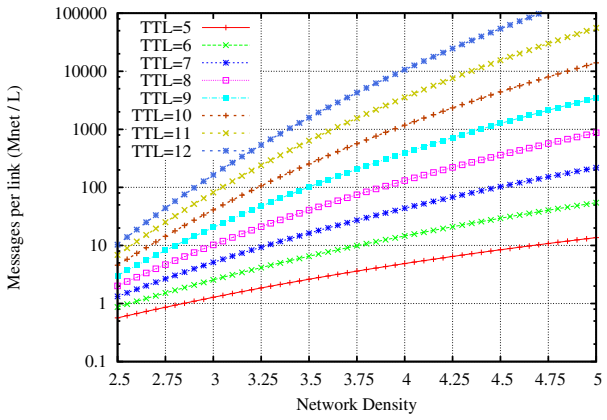


Fig. 2. Evaluation of the message generation for a single flood according to the network density and TTL bound (reduced in messages per link)

TABLE I. TOPOLOGICAL PROFILES PARAMETERS FOR ANALYTIC ESTIMATIONS

	Topological Description	Density (δ_{net})	Diameter (D)
Profile 1	"Scarce Extended"	2.8	10
Profile 2	"Medium"	3.5	7
Profile 3	"Dense Little"	5	5
Profile 4	"Medium Extended"	3.5	10
Profile 5	"Dense Medium"	5	7
Profile 6	"Dense Extended"	5	10

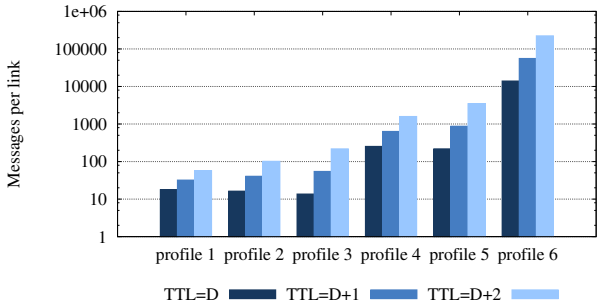


Fig. 3. Message generation for a single flood for several network profiles and TTL arrangements

III. ANALYTIC OVERHEADS ESTIMATIONS

In this section we begin by estimating LOCARN overheads through the analysis of the non data messages generated by a single flood. Then we exhibit a method to analyse the OAM cost, and present numerical results for several network profiles according to the number of services present in the network.

A. Enhanced Flooding Messages Estimation

The two parameters determining the message generation of a flooding process bounded by a Time To Live (TTL) are the network density (δ_{net}) and the TTL value. Whether a network having N nodes and L links, the mean density corresponds to $\delta_{net} = L \times 2/N$ (the mean number of interfaces per node). The average message generation can be estimated quite easily: the origin node sends on average δ_{net} messages (across all interfaces), whereas successive nodes retransmit on average $\delta_{net} - 1$ until the TTL is reached (across all

interfaces except the incoming one). Thereby, whether M_N representing the number of messages generated by duplications at the propagation step N, we have:

$$M_N = \delta_{net} \times (\delta_{net} - 1)^{N-1}$$

Then we get an estimation of the global amount of messages generated over the network (M_{flood}) by summing terms from M_1 to M_{TTL}:

$$\begin{aligned} M_{flood}(\delta_{net}, TTL) &\approx \delta_{net} \times \sum_{i=1}^{TTL} (\delta_{net} - 1)^{i-1} \\ &\approx \delta_{net} \times \frac{(\delta_{net} - 1)^{TTL} - 1}{\delta_{net} - 2} \end{aligned} \quad (1)$$

This expression provides an *upper bound* estimation in so far it does not include that, along the flooding propagation, messages which achieved a cycle are discarded. Hence all messages estimated subsequently to loop detections are counted in excess, which yet allows to express the message generation *magnitude* through a simple formula. In Fig.2 are gathered numerical applications giving an overview of this magnitude reduced per link, by varying TTL (ranging from 5 to 12) and δ_{net} (ranging from 2.5 to 5). These results correspond to a numerical application of (1) to a network with $N = 50$ nodes[6], dividing the global amount of messages generated over the network by the number of links ($L = (\delta_{net} \times N)/2$). Unsurprisingly the amount of messages quickly explodes with the network density, making the overhead cost unacceptable for "dense networks" and "excessive" TTL range. In Fig.3 are reported the magnitudes of the messages generation for one flood that can be expected per link upon several networks (profiles are exposed in Table I). The TTL of flooding is chosen to exceed the diameter in order to estimate the most expensive service path discovery. The obtained results bring out that upon "reasonable" network dimensions (profiles 1, 2 and 3) a link transmits on average between ten and hundred of path request messages within a flooding process.

B. Overheads Estimation Over Three Network Profiles

As exposed before, services involve floods for their establishment and periodically for their possible adaptations, while active paths are continuously checked by OAM exchanges. We call Optimization Interval (OI) the period of path optimizations, and Service Check Interval (SCI) the period of OAM exchanges. To estimate the OAM overhead, we have to consider the amount of active services (S), and the SCI interval. Yet the OAM message generation also depends on the active paths lengths: if in a network all declared services are using ten hops paths, the OAM exhanges from end-to-end will consume twice of the network resources as if they would be using five hops paths. Hence, the exact overhead generated by OAM exchanges among a period of time actually depends on the active paths lengths during this period. Hence, to get an estimation of the OAM overhead for S services, we have to make significant assumptions about the active path lengths. For a meaningful estimation, we assumed that the services' endpoints "remoteness" (i.e. their shortest paths in number of hops) distribution follows a gaussian distribution probability from 1 to D hops. Then, the effective selected paths can vary over time among the shortest path and the TTL value according to the successive optimizations in the paths selections. By observing that LOCARN active paths lengths rarely exceed the shortest path over two hops through simulations of a realistic transport network topology and traffic matrix, we are able to give an upper bound estimation of OAM messages per second and per link according to S and SCI.

We provide in Table II the estimations of LOCARN overheads due to non-data packets, gathering estimations of floods overhead based on the III.A formula and estimations of OAM overhead. Results are reported for three network topological profiles. For each profile, the LOCARN settings (TTL, OI, SCI) are declined together with the amount of active services across the network. The TTL values are arranged to exceed the network diameter ($TTL = D + 1$, $TTL = D + 2$) providing an important potential for paths optimization. S, OI and SCI are chosen to expose orders of magnitudes ($S = 100/1000$; $OI = 10/30s$; $SCI = 10/100ms$). The last column reports the sum of overheads estimations per link in ratio beside 1Gb/s that is used here as an indicative basis datarate which allow transpose estimations for higher granularities. The numerical applications confirm that in absolute, the LOCARN overhead due to non-data packets is substancial: on average several mega bit per second and per link for a thousand services. However, in a Transport Network context, such overheads represent relatively few bandwidth consumption. At most, for a thousand of highly resilient ($SCI = 10ms$) and a highly adaptive ($OI = 10s$), the cumulated overheads reach on average 0.85%, 1.98% and 1.50% of 1Gb/s links respectively for profile 1, 2 and 3. Given that nowadays the Transport Networks transmission capacities are commonly around 10, 40 or 100Gb/s in core segments, a such overhead magnitude remains quite acceptable (notwithstanding, the bandwidth consumption estimated assume the packets transfer in the two links directions, hence the effective consumption on average corresponds actually to half of the last column).

Finally, these analytic results permit to point out that the real scalability issue of LOCARN in terms of overheads' mean bandwidth consumption in a transport network context relies on the fact that both for control and OAM, the non-data packets generation linearly depends on the amount of services (S) declared in the network. This is actually the consequence of the "service oriented" design of LOCARN (i.e. where each service involves its own generation of packets for routing and OAM).

IV. Simulation Results Upon a Core Network Example

In this section, we expose some LOCARN's simulation results. The architecture mechanisms (data plane, control plane and OAM) have been implemented in the Omnet++ discrete event simulator. A network infrastructure is simulated by taking into account: the network topology, the links datarates and the propagation delays (packet-processing delays are not implemented here as they are negligible by report to the propagation ones in the example taken, see [19]). To study performances and costs over a concrete and realistic use case as those encountered in Orange Group, the results of this section correspond to a LOCARN application to the national (France)

[6]application for a 100 nodes with the same density gives similar magnitudes, globally low

TABLE II. ANALYTIC ESTIMATIONS OF NON-DATA PACKETS OVERHEADS PER LINK FOR THREE NETWORK PROFILES

Network profiles	TTL	S	OI	SCI	Floods overheads	OAM overheads	Cumulated overheads	Ratio 1Gb/s
Profile 1	12	100	10s	10ms	537Kbps	310Kbps	847Kbps	**0.08%**
				1s		3Kbps	540Kbps	**0.05%**
			30s	10ms	179Kbps	310Kbps	489Kbps	**0.05%**
				1s		3Kbps	182Kbps	**0.02%**
		1000	10s	10ms	5367Kbps	3108Kbps	8475Kbps	**0.85%**
				1s		31Kbps	5398Kbps	**0.54%**
			30s	10ms	1789Kbps	3108Kbps	4897Kbps	**0.49%**
				1s		31Kbps	1820Kbps	**0.18%**
Profile 2	9	100	10s	10ms	212Kbps	1764Kbps	1976Kbps	**0.20%**
				1s		18Kbps	230Kbps	**0.02%**
			30s	10ms	71Kbps	1764Kbps	1835Kbps	**0.18%**
				1s		18Kbps	89Kbps	**0.00%**
		1000	10s	10ms	2116Kbps	17644Kbps	19760Kbps	**1.98%**
				1s		176Kbps	2292Kbps	**0.23%**
			30s	10ms	705Kbps	17644Kbps	18349Kbps	**1.83%**
				1s		176Kbps	881Kbps	**0.09%**
Profile 3	7	100	10s	10ms	1328Kbps	169Kbps	1497Kbps	**0.15%**
				1s		2Kbps	1330Kbps	**0.13%**
			30s	10ms	443Kbps	169Kbps	612Kbps	**0.06%**
				1s		2Kbps	445Kbps	**0.04%**
		1000	10s	10ms	13284Kbps	1688Kbps	14972Kbps	**1.50%**
				1s		3Kbps	540Kbps	**0.05%**
			30s	10ms	4428Kbps	1688Kbps	5916Kbps	**0.59%**
				1s		3Kbps	4431Kbps	**0.44%**

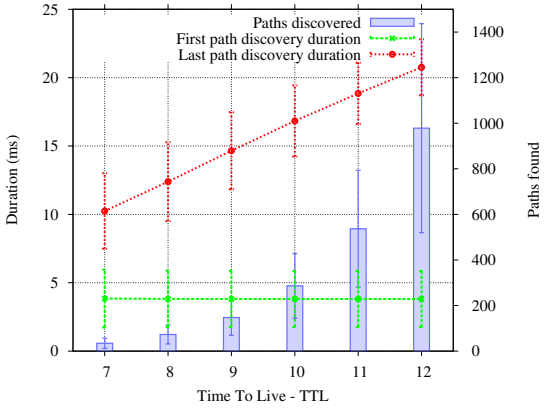


Fig. 4. Path discovery performances (average and standard deviation for 1000 randomly picked services)

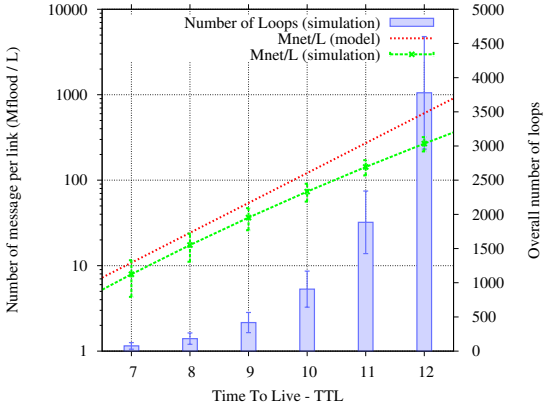


Fig. 5. Messages generated per link for a single flood: model and simulation

core network topology. The latter includes N=42 nodes, L=68 links, has a diameter D=7 hops and a mean density $\delta_{net} = 3.23$ whereas the nodes degree standard deviation is $\sigma_\delta = 1.12$. Propagation delays are implemented in accordance with the optical propagation across the real inter-nodes distances (the mean propagation delay is $445.9\mu s$). Nodes interfaces are associated with unlimited FIFO queues without priorization between data, control or OAM packets.

A. Path Discovery Performances and Overhead Cost

Hereafter we bring out the performances and overhead cost of the path discovery process, namely the number of paths discovered and the discovery duration beside the amount of overhead generated by the flooding discovery.

To get a statistical overview of the path discovery performances (Fig.4), we pick up 1000 services randomly among the topology: one random node as origin and a distinct random node as destination. Then we launch discoveries from each origin, varying TTL from 7 to 12 (which corresponds here to TTL=D and TTL=D+5). The answer packets arrival to the origin are counted and their arrival time is noticed to respectivelly get the number of paths discovered and the discoveries durations. Finally, the figure provides the first and last path discovery duration through the average and standard deviation (respectively through green and red lines), refer to the left axis for durations. The discovered paths are expressed by histograms (read on the right axis). We must also note that the significant standard deviations (both for durations and number of paths) is due to the fact that the random picking of services give a significant variance of enpoints "remotess" which is itself determining on the path discoveries.

To get a statistical overview of the path discovery overhead cost (Fig.5), we launch this time one path discovery process for each of the 42 nodes and observe each time the message generation, varying TTL from 7 to 12. The overall generated messages during each flood (i.e. the request messages) are counted. Finally the figure reports the average number of flood messages reduced per link (green line); the previous analytic expectation (see section III.A) is reported for comparison (red line). The simulation confirms that the analytic formula provides an upper bound estimation which is relatively accurate for this network (due to the low standard deviation of nodes degrees, $\sigma_\delta = 1.12$). Yet, it is less accurate when the TTL

deviates too much from the network diameter because the amount and size of loops become too significant to be ignored in the analysis. The number of detected loops is reported with the histograms (read on the right axis).

By crossing the Fig.4 and Fig.5 results, it is possible to bring out the relevance of TTL settings on the simulated network. For example with TTL=10, path discoveries provide on average hundreds of paths. If we assume for example that a such TTL setting gives sufficient amount of paths for the expected potential of services adaptation and recovery under failure. By looking at the discoveries durations, we see that the last path is obtained around 17ms whereas the first path is still obtained around 5ms independently from the TTL. Under such discovery durations, a sub 50ms recovery of a broken path can be achieved if faults can be detected between 30ms and 45ms (typically, OAM should be sent every 10ms and a fault considered upon the lake of three packets). On the other hand, in terms of overhead cost, with TTL=10 each flood generates on average almost 70 packets per link.

B. The Maximum Impact of Overlapping Floods

As we have seen, floods due to path discoveries generate numerous packets across the network, consuming bandwidth. Until now we have studied the number of packets generated, and the involved overhead cost in terms of its per link and per flood bandwidth global consumption over time (Table II). Yet, beyond the amount of packet generated, the critical aspect of floods' message generation is to occur along a very short period of time (which is actually related to the infrastructure's propagation delays). Consequently, flooding does not constitute regular overheads but produce peaks of bandwidth consumption. Following the previous subsection example, if a flooding with $TTL = 10$ involves on average 70 packets to be transfered per link (i.e. in the two direction); thus an interface belonging to the flooding domain will have to cope on average with the reception of 35 packets during 17ms, i.e. almost 2060packets/s during this period. Moreover, overhead peaks may cumulate if several path discoveries are overlapped (i.e. occur in the same time). In normal conditions, flooding is launched periodically for "path optimizations". Statistically, the amount of overlapping discovery processes is low in normal conditions given the envisioned optimization interval and the duration of discoveries. However, in case of failure (node or link), when N services are disrupted, N discovery processes are triggered across the network from the disrupted services' origin nodes. If the detection time is tuned very quick, the probability of overlapping becomes very high. The overlapping of the N floodings messages generation may cause the filling of packets queues, and even impact the Quality Of Service (QoS) of data transmission, in particular jitter[7]. Yet the impact of floods overlapping is difficult to estimate in a comprehensive way because it may vary a lot according to: (i) the spreading of floods starting over time; (ii) links' propagation delays; (iii) links' available bandwidths during floods propagation.

With Fig.6 and Fig.7, we exhibit the impact of N overlapping floods in terms of queues load and jitter. To bring out some meaningful estimations, we consider a very worst case scenario, providing the very maximum impact that can be expected on the network involving N overlapped discovery proccesses with different TTL. We proceed as follows: all links capacities are set to 1Gb/s (that we use as an indicator like in Table II). A constant data traffic is sent over a path "especially at risk"[8] in the network up to two-third of capacities. Then we launch *at the same time*[9] N discovery processes (N=20,30,40,50) by varying TTL from 7 to 12. We report in Fig.6 the maximum queue length that have been reached all along the studied path, until the end of flooding sessions and in Fig.7 the worst packet end-to-end jitter[10] along the data path. To bring out impacts independently from the starting points, we launch 20 simulations involving randomly picked origins samples and report the averages and standard deviations.

Finally in Fig.8 and Fig.9 we observe the evolution of queuing length over time along the previous sensible path in the case all nodes (42) would launch a flood at the same time, in case of recovery for example. Figures depict the

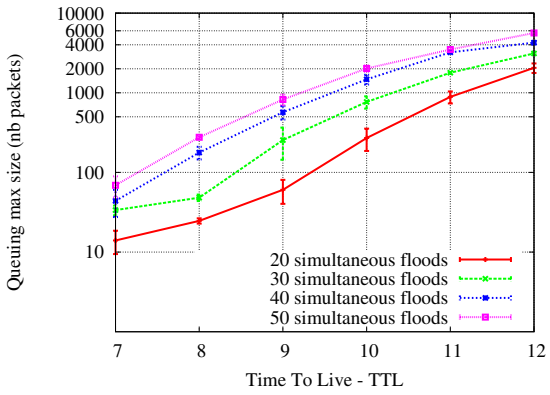


Fig. 6. Queue maximum length along the studied path for N simultaneous floods over the network (1Gb/s links)

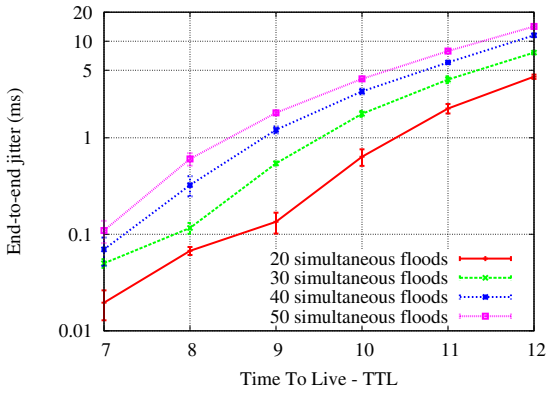


Fig. 7. Worst data packet jitter due to additionnal queuing loads along the studied path for N simultaneous floods over the network (1Gb/s links)

[7]as we consider a same priority level both for data and control packets relaying for the sake of simplicty

[8]such a path have been identified as exposed to flooding perturbations over several simulations: it is a long path passing through the dense network zone

[9]the fact than N floods are launched strictly at the same time actually corresponds to a very worst case that statistically should never happen. This allow us to bring out an upper bound of impacts on the network.

[10]by substracting arrival from sending times we get all packets delays. The worst packet jitter is simply obtain in the simulator by substracting the longest delay reached over the simulation by the shortest one.

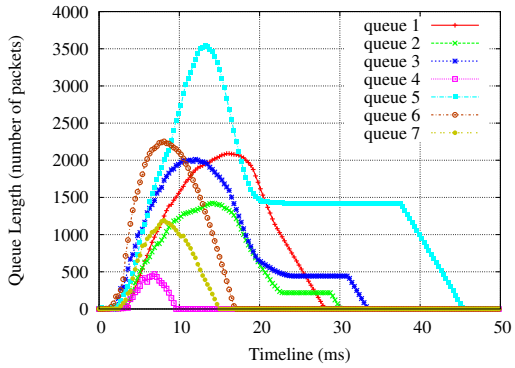


Fig. 8. Evolution of queing load along the studied path for 1Gb/s links, N=42 simultaneous path discoveries, TTL=12

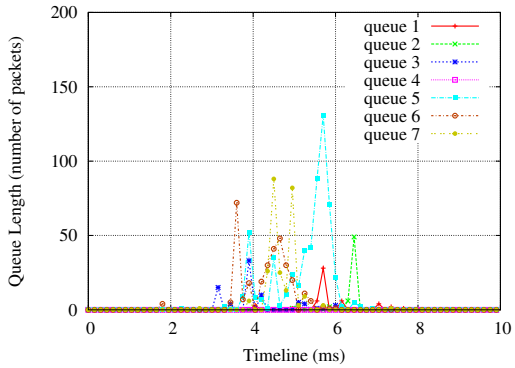


Fig. 9. Same scenario than Fig.8 with 10Gb/s links

evolution of queues lengths over time along the studied path. Respectively Fig.8 and Fig.9 permit to show how much the impact is reduced by passing from 1Gb/s to 10Gb/s links.

V. Conclusion

We have described LOCARN: an alternative packet architecture providing point-to-point communications through a very simple design and concepts. Beyond the LOCARN presentation, this paper quantify the generated overheads which constitute its critical aspect. The obtained results point out that in a meshed transport network context, it is possible within LOCARN to provide thousands of very adaptive and very resilient communications for acceptable overheads costs. Meanwhile, in the very extreme scenario of floods overlapping on the simulated example, beyond 10Gb/s links transmission capacities, the bursts of tens of simultaneous recoveries becomes imperceptible. Consequently, LOCARN represents a very interesting alternative to the existing protocols. Among several potentials for application, we think that LOCARN constitutes an anticipated answer to operators futures needs for dynamicity in their meshed core networks: providing very simply some adaptative and resilient communications without requiring plannification or preliminary configured protection mechanisms.

References

[1] D. Awduche, L. Berger, D. Gan, T. Li, V. Srinivasan, and G. Swallow, "RSVP-TE: Extensions to RSVP for LSP Tunnels," RFC 3209 (Proposed Standard), Internet Engineering Task Force, Dec. 2001, updated by RFCs 3936, 4420, 4874, 5151, 5420, 5711, 6780, 6790. [Online]. Available: http://www.ietf.org/rfc/rfc3209.txt

[2] P. Pan, G. Swallow, and A. Atlas, "Fast Reroute Extensions to RSVP-TE for LSP Tunnels," RFC 4090 (Proposed Standard), Internet Engineering Task Force, May 2005. [Online]. Available: http://www.ietf.org/rfc/rfc4090.txt

[3] L. Andersson, I. Minei, and B. Thomas, "LDP Specification," RFC 5036 (Draft Standard), Internet Engineering Task Force, Oct. 2007, updated by RFCs 6720, 6790. [Online]. Available: http://www.ietf.org/rfc/rfc5036.txt

[4] T. Li and H. Smit, "IS-IS Extensions for Traffic Engineering," RFC 5305 (Proposed Standard), Internet Engineering Task Force, Oct. 2008, updated by RFC 5307. [Online]. Available: http://www.ietf.org/rfc/rfc5305.txt

[5] D. Katz and D. Ward, "Bidirectional Forwarding Detection (BFD)," RFC 5880 (Proposed Standard), Internet Engineering Task Force, Jun. 2010. [Online]. Available: http://www.ietf.org/rfc/rfc5880.txt

[6] T. L. Rodeheffer, C. A. Thekkath, and D. C. Anderson, "Smartbridge: A scalable bridge architecture," *SIGCOMM Comput. Commun. Rev.*, vol. 30, no. 4, pp. 205–216, Aug. 2000. [Online]. Available: http://doi.acm.org/10.1145/347057.347546

[7] N. Varis, J. Manner, M. Srel, and T. Kiravuo, "Dbridges: Flexible floodless frame forwarding," *Computer Networks*, vol. 57, no. 17, pp. 3601 – 3616, 2013. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S1389128613002636

[8] R. Perlman, "Rbridges: transparent routing," in *INFOCOM 2004. Twenty-third AnnualJoint Conference of the IEEE Computer and Communications Societies*, vol. 2, March 2004, pp. 1211–1218 vol.2.

[9] E. Mannie, "Generalized Multi-Protocol Label Switching (GMPLS) Architecture," RFC 3945 (Proposed Standard), Internet Engineering Task Force, Oct. 2004, updated by RFC 6002. [Online]. Available: http://www.ietf.org/rfc/rfc3945.txt

[10] F. Paolucci, F. Cugini, A. Giorgetti, N. Sambo, and P. Castoldi, "A Survey on the Path Computation Element (PCE) Architecture," *IEEE, Communications Surveys Tutorials*, vol. 15, no. 4, pp. 1819–1841, Fourth 2013.

[11] "The TIGER2 project deliverables." [Online]. Available: http://projects.celtic-initiative.org/tiger2/deliverables.htm

[12] S. Ghamri-Doudane, L. Ciavaglia, D. Rossi, R. Casellas, R. Martinez, and R. Munos, "D40, network control and management evolution towards autonomic networking including study cases definition," CELTIC TIGER 2, Tech. Rep., 2010. [Online]. Available: http://projects.celtic-initiative.org/tiger2/TIGER2_D40.pdf

[13] S. Ghamri-Doudane, L. Ciavaglia, D. Rossi, L. Fabrega, R. Clavier, C. Simon, and P. Varga, "D41-2, design of study case solution & study cases evaluations," CELTIC TIGER 2, Tech. Rep., 2010. [Online]. Available: http://projects.celtic-initiative.org/tiger2/TIGER2_D41_2.pdf

[14] Z. Movahedi, M. Ayari, R. Langar, and G. Pujolle, "A survey of autonomic network architectures and evaluation criteria," *Communications Surveys Tutorials, IEEE*, vol. 14, no. 2, pp. 464–490, Second 2012.

[15] E. Rosen, D. Tappan, G. Fedorkow, Y. Rekhter, D. Farinacci, T. Li, and A. Conta, "MPLS Label Stack Encoding," RFC 3032 (Proposed Standard), Internet Engineering Task Force, Jan. 2001, updated by RFCs 3443, 4182, 5332, 3270, 5129, 5462, 5586. [Online]. Available: http://www.ietf.org/rfc/rfc3032.txt

[16] D. Johnson, Y. Hu, and D. Maltz, "The Dynamic Source Routing Protocol (DSR) for Mobile Ad Hoc Networks for IPv4," RFC 4728 (Experimental), Internet Engineering Task Force, Feb. 2007. [Online]. Available: http://www.ietf.org/rfc/rfc4728.txt

[17] G. Rossini, D. Rossi, C. Betoule, R. Clavier, and G. Thouenon, "Fib aplasia through probabilistic routing and autoforwarding," *Computer Networks*, vol. 57, no. 14, pp. 2802–2816, 2013.

[18] C. Betoule, T. Bonald, R. Clavier, D. Rossi, G. Rossini, and G. Thouenon, "Adaptive probabilistic flooding for multipath routing," in *New Technologies, Mobility and Security (NTMS), 2012 5th International Conference on*. IEEE, 2012, pp. 1–6.

[19] R. Ramaswamy, N. Weng, and T. Wolf, "Characterizing network processing delay," in *Global Telecommunications Conference, 2004. GLOBECOM'04. IEEE*, vol. 3. IEEE, 2004, pp. 1629–1634.